

Memoria de Práctica 5: Colecciones, Genéricos, Lambdas y Patrones de Diseño

Autores: Adrián Gómez y Javier Agüero

Esta práctica consiste en diseñar e implementar nuestro código para construir árboles de decisión aplicando los conceptos de colecciones, genericidad, lambdas y patrones de diseño. Además, la idea de nuestro código no es solo basada en la funcionalidad, sino también en tenerlo de forma limpia, bien estructurada y escalable. Es por ello, que a lo largo de la práctica hemos ido construyendo todo por apartados, empezando por la creación de datasets y la extracción de sus características, siguiendo por la creación de árboles de decisión, ...

A nivel de diseño, nos encontramos con la implementación de la interfaz `Featurizer` que nos permite desacoplar la extracción de las características de los objetos, consiguiendo así que nos sea más fácil poder extenderlo a otros tipos de datos (en nuestro caso usamos objetos de tipo `Person` y `PersonFeaturizer` para las características, para probar el funcionamiento de nuestro código en los tests). Luego, tenemos las clases `Dataset` y `Feature`, encargadas del almacenamiento de objetos y sus respectivas características, y de la clase `DecisionTree` (que tiene como clases internas `Node` y `Branch` para facilitar su implementación) basada en el uso de predicados y expresiones lambda.

Para dotar a nuestra librería de la capacidad de inferir árboles de decisión a partir de datos, extendimos nuestro modelo base creando la clase `LabeledDataset` (que hereda de `Dataset`) y la interfaz `LabelProvider`. Esto nos permite almacenar objetos junto a sus etiquetas de clasificación. Con esta estructura, implementamos la clase `GreedyTreeLearner`, la cual aplica un algoritmo voraz (*greedy*) para construir el árbol de manera recursiva, dividiendo el conjunto de datos iterativamente hasta que todos los objetos compartan la misma etiqueta o se agoten las características disponibles.

Para hacer que el algoritmo de aprendizaje sea flexible y extensible, aplicamos el patrón de diseño `Strategy`. Extraímos la lógica de decisión de la "mejor característica" a una interfaz genérica `FeatureSelectionStrategy`. De este modo, `GreedyTreeLearner` delega esta responsabilidad, permitiendo instanciar el aprendiz con diferentes heurísticas sin modificar

su código interno. Implementamos dos estrategias concretas: RandomFeatureStrategy (que elige una característica al azar) y MisclassificationStrategy (que evalúa todas las características y selecciona la que produce el menor error de clasificación).

Finalmente, para proporcionar mecanismos de visualización sin acoplar la lógica de presentación a la estructura del árbol, implementamos el patrón de diseño Visitor. Definimos la interfaz DecisionTreeVisitor y la interfaz DecisionTreeElement (implementada por DecisionTree, Node y Branch mediante el método accept). Gracias al *doble dispatch*, podemos recorrer el árbol y generar diferentes representaciones. Implementamos dos visualizadores concretos: IndentedTextVisitor, que genera un esquema en texto plano tabulado, y GraphvizVisitor, que genera código en formato DOT para su renderización gráfica. Esta arquitectura garantiza que añadir nuevos formatos de exportación en el futuro sea trivial y no requiera tocar las clases del árbol.